

Happy Shopping — Master Project Reference

Save this file locally. Paste its contents into any new Claude or Claude Code conversation to restore full context.

Branding note: the overall platform is "Happy Shopping." The Kids vertical keeps its own brand, "Happy Baby" (parallel to "Happy Men" and "Happy Women"). Repo names were intentionally left unchanged (still `happy-baby`, `happy-baby-app`, etc.) — only user-facing text and package/doc names were rebranded.

1. Architecture Overview

Web app (`happy-baby` repo — Sireesh441/happy-baby, rebranded to "Happy Shopping")

- Next.js + Tailwind, deployed on Vercel (production)
- Database: PostgreSQL (Supabase), via Prisma
- Auth: NextAuth (web) + custom JWT endpoints for mobile (`/api/mobile-auth/login`, `/signup`, `/me`)
- Payments: Razorpay (test mode integrated)
- AI: Anthropic API — "Ask Happy Baby" shopping assistant chat widget
- Multi-vertical: Kids ("Happy Baby"), Men ("Happy Men"), Women ("Happy Women") — root (`/`) is a 3-card hub
- API layer: REST routes under `/api/products`, `/api/cart`, `/api/orders` — public GETs, authenticated writes
- CORS: configured to allow the mobile app to call these APIs directly

Mobile app (`happy-baby-app` repo)

- Expo (React Native), SDK 54 (downgraded from 57 for Expo Go compatibility)
- Connects to the same live backend APIs as the web app (single source of truth)
- Home: debounced search + suggestions, Kids/Men/Women vertical selector
- Shop screen: Instamart-style two-pane layout (category rail left, product grid right)
- Product detail: image carousel, sticky Add to Cart bar, "You might also like"
- Cart: persistent guest cart via AsyncStorage (survives restart, no login required)
- Auth: JWT-based, stored via expo-secure-store (native) / AsyncStorage fallback (web), required only at checkout
- Checkout: in progress (Razorpay React Native SDK)

Known environment notes

- Windows dev machine — PowerShell execution policy had to be set to RemoteSigned
- npm had a Windows-specific bug with `create-expo-app`'s dry-run JSON parsing — fixed by downgrading to npm@8

- Prisma requires `postinstall: prisma generate` in package.json for Vercel builds to succeed
 - DATABASE_URL must use Supabase's pooled connection (port 6543, `?pgbouncer=true`); DIRECT_URL (port 5432) used only for migrations
-

2. Progress Log (Days Completed)

Day	Status	Notes
1	✓	Node, VS Code, Git, Claude Code installed; Next.js project created
2	✓	Homepage: header, hero, category tiles, footer
3	✓	/shop catalog with filters, 20 dummy products
4	✓	Product detail page, "You might also like"
5	✓	Persistent cart (web)
6	✓	Auth (NextAuth, web)
7	✓	Checkout + Razorpay (web)
8	✓	Admin panel, DB-backed product CRUD
9	✓	AI shopping assistant (web)
10	✓	Migrated SQLite → PostgreSQL (Supabase), deployed to Vercel, fixed CORS + localhost-fetch production bug
—	✓	Multi-vertical expansion (Kids/Men/Women), 24 dummy products added
11 (app)	✓	Expo setup, home screen, search, vertical selector
12 (app)	✓	Product detail screen, sticky Add to Cart bar
13 (app)	✓	Persistent guest cart, verified on native device
14 (app)	✓	Mobile auth (JWT), verified on native device
15 (app)	⏸ Paused	Virtual Try-On — HF integration had garment-misclassification bug (jeans rendered as shirt). Decision: revisit with self-hosted GPU later; feature hidden behind flag for now
16 (app)	➡ Next SOON	Checkout + Razorpay React Native SDK
17 (app)	Planned	Order history, wishlist, AI assistant chat (mobile)

Day	Status	Notes
18	Planned	Polish, test builds (APK/TestFlight) (app)

3. Differentiator Feature Roadmap ("Fit Certain")

Validated via research: industry data confirms sizing causes ~52% of apparel returns; combining visual try-on + data-driven fit prediction is considered the current best-practice combination for cutting returns. No competitor (Myntra/Ajio/etc.) combines this with a **family-wide, multi-vertical account** — which is Happy Baby's structural advantage.

Layer 1 — Enhanced Virtual Try-On

- Static photo try-on (paused, see Day 15 above)
- Stretch goal: 360°/multi-angle try-on — see feasibility note below

Layer 2 — Fit Confidence Score

- % match score per product based on saved measurements + crowd-sourced "true to size" feedback
- Start simple (rule-based against saved measurements), refine with real order/return data later

Layer 3 — Family Fit Profiles (the core differentiator)

- One account, multiple family member profiles (parents + kids)
- Age-based growth tracking for kids ("Grows With You" sizing — reminds parents when it's time to size up)
- Any vertical (Kids/Men/Women) already knows who you're likely shopping for

Layer 4 — Proof-Locked Returns

- Timestamped, tamper-evident photo/video at packing AND unboxing
- Transparent step-by-step refund status tracker, refunds to original payment method by default (not wallet-locked)
- Directly targets the #1 and #2 complaint themes found across Myntra/Ajio reviews (unfair return rejections, refund black-holes)

360° try-on — feasibility note True continuous 3D try-on needs a 3D body mesh + garment simulation (heavy, specialized). A realistic middle ground: generate try-on images at ~6-8 fixed angles (front, 3/4, side, back) and present them in a spin-viewer (like product-photo 360 spinners). This multiplies API/inference cost by ~6-8x per try-on vs a single image — worth prototyping with 1 user before committing, and a strong reason to prioritize the self-hosted GPU path (Day 15 follow-on) once volume matters, since per-call third-party API costs would add up fast at this multiplier.

4. Gen-Z Design Considerations

- Short, punchy copy over dense text; avoid corporate tone in UI copy and the AI assistant's voice
- Vertical, scrollable product discovery (reels-style browsing) resonates more than static grids alone
- Social proof front and center: ratings, review counts, "X people bought this today" — already partially in place via badges
- Fast checkout matters more than feature depth — every extra step loses users
- Dark mode as a first-class option, not an afterthought
- BNPL (Buy Now Pay Later) options are increasingly expected, not a bonus
- UGC/community angle — could tie into the AI assistant later (e.g. styling from real customer photos, with consent)

5. Cost Reduction Notes (API + Storage)

- **AI assistant:** cache common queries/responses where reasonable; consider a cheaper/faster model for simple product lookups vs. complex styling questions
- **Images:** use a CDN with compression (Cloudinary free tier covers early-stage volume); lazy-load product images
- **Database:** Supabase free tier is fine for now; monitor connection pooling to avoid overages
- **Try-on:** this is the biggest cost driver — gate it behind login (already planned) and consider a daily/monthly free-use cap per user once live
- **Vercel:** watch serverless function execution time/count on the free/hobby tier as traffic grows

6. Productizing Features as Standalone Sellable Products

Once proven inside Happy Baby, several of these could be offered as B2B SaaS/API products to other Indian D2C brands:

- **Fit Confidence Score API** — "sizing-as-a-service" for any fashion e-commerce site
- **Proof-Locked Returns module** — return-fraud/dispute-prevention SaaS, sold independently
- **Virtual Try-On API** — white-labeled for other stores
- **AI Shopping Assistant widget** — embeddable chat widget for other e-commerce sites

This is a later-stage consideration — prioritize proving these work well inside Happy Baby first before packaging externally.

7. Immediate Next Step

Day 16 — Checkout + Razorpay React Native SDK integration in the mobile app.